

提出背景

RV控制软件由三个主要模块组成：（1）**传感器模块**：定期收集来自机载传感器的实时输入信号；（2）**控制逻辑模块**：处理传感器数据、参考状态和任务需求来生成适当的控制信号；（3）**通信模块**：与地面控制站保持双向通信。它接收用户命令，同时传输车辆状态、任务进度和反馈，支持远程监控和协调操作。

RV控制软件接收的输入主要包括三类：（1）配置文件中的配置参数；（2）来自任务文件的任务命令；（3）来自传感器的环境数据。

不同类型的输入可能影响同一个系统变量。例如，配置参数MOT_THST_EXPO调整油门曲线，而控制命令MAV_CMD_DO_THROTTLE直接设置油门百分比。因此，当RV接收到多个输入时，这些输入可能影响相同的物理量。冲突效应可能导致系统状态不一致。此外，由于输入可以直接修改关键数据，即使单个输入也可能触发异常行为。

提出背景

RV软件中可配置参数、命令输入和环境感知数据构成的**巨大参数空间**存在安全风险。

主流的模糊测试技术难以很好地解决这一问题。

(1) **RVFuzzer**: 通过二分查找和策略组合等技术减少冗余测试输入，但是无法控制在测试期间实际执行了代码的哪些部分，因此不清楚是否充分探索了具有潜在漏洞模式的代码区域；

(2) **PGFuzz**: 使用从开发人员文档中提取的有限的一组基于规则的策略来发现漏洞，但是遗漏了大量控制计算公式中的漏洞，特别是那些未被记录的漏洞。

威胁模型

由于RV软件中的许多配置参数和控制命令是**用户可配置**的，赋值语句漏洞的存在会使系统对错误配置变得敏感。用户可能会无意中设置超出典型或合理边界的值，从而触发非预期行为并危及RV的安全。

此外，攻击者有一定可能性获得**RV软件的控制权**。此类攻击可能包括GCS欺骗或通信链路劫持等策略。一旦获得控制权，攻击者可能会尝试发出破坏性命令，例如禁用油门输出、强制无人机着陆或解除武装。虽然这些动作可能是有效的，但它们通常容易被检测到，并且**可以被阻止**。

相比之下，赋值语句实现中的漏洞可能允许攻击者以更隐蔽的方式干扰系统行为。这为破坏提供了一个机会，使用传统的监控工具很难检测到。

ADGFUZZ介绍

ADGFUZZ：专门用于检测RV控制软件中赋值语句漏洞，仅对影响赋值操作的RV输入参数进行模糊测试。

赋值语句漏洞：RV控制软件的源代码将各种物理公式和数学模型编码到程序逻辑中，这通常涉及赋值计算，很容易出现参数赋值不完整或不符合公式要求的情况，从而增加了出现漏洞的可能性。

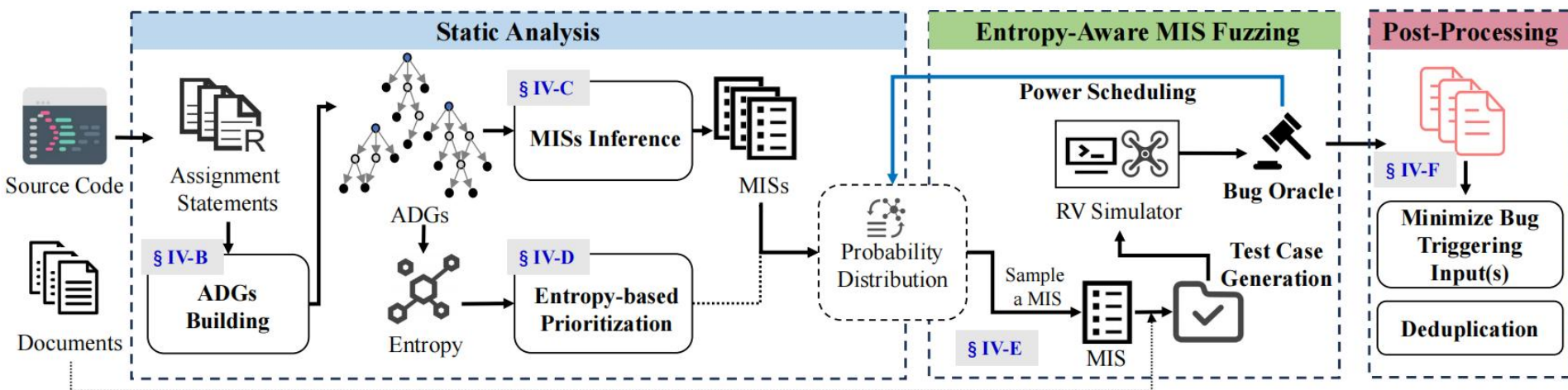
RV控制软件中的赋值语句在计算过程中涉及许多有固定数据依赖关系的并且遵循一致的命名约定的变量，包含丰富的语义信息（这些变量通常与RV输入参数的命名模式保持一致）。

利用命名相似性来识别与特定赋值语句相关的RV输入参数，并对它们进行有针对性的模糊测试。

ADGFUZZ的原理

静态分析阶段：用赋值依赖图（ADG）捕获RV源代码中变量之间的相互依赖赋值关系。

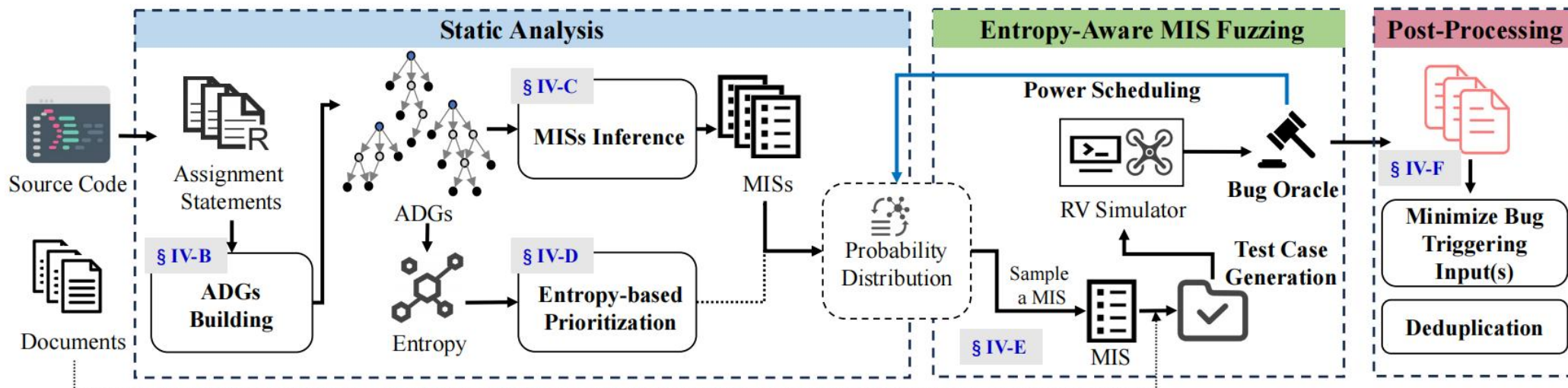
- （1）在每个ADG中，我们识别叶变量（LVs）即不依赖于任何其他赋值的变量；
- （2）我们应用细粒度的、基于术语的语义匹配，将每个LV映射到相应的RV输入参数子集。我们将这些子集称为匹配输入集（MISs）；
- （3）特定赋值语句与MIS之间的数据依赖关系被明确构建出来。对MIS生成测试用例使得ADGFUZZ专注于输入空间中更小、更容易出错的部分，从而显著提高了赋值语句漏洞检测的效率和有效性。



ADGFUZZ的原理

模糊测试阶段：

- (1) 基于ADG计算每个MIS的多种类型的熵，执行熵感知的MIS模糊测试，优先考虑那些具有较高漏洞发现潜力的MIS，从而提高漏洞发现的效率；
- (2) 执行漏洞去重以消除冗余报告。



ADGFUZZ的原理

赋值语句：一个元组 $S = (y, X, O)$ ，其中：

- (1) $y \in V_{\text{dep}}$ ，这里 V_{dep} 是赋值表达式左侧（LHS）的依赖变量集合；
- (2) $X = \{x_1, \dots, x_n\} \subseteq (V_{\text{ind}} \cup F)$ 是表达式语句右侧（RHS）的独立变量（操作数）的集合， F 表示函数调用；
- (3) $O \subseteq \{\oplus_1, \dots, \oplus_m\}^{n-1}$ ，其中 $\oplus_i \in \underbrace{\{+, -, \times, \div\}}_{\text{arithmetic}} \cup \underbrace{\{\wedge, \vee, \neg, =\}}_{\text{logical}}$

赋值依赖图（ADG）是由函数内一个或多个相互依赖的赋值语句构建而成的。

ADGFUZZ的原理

赋值依赖图：一个赋值依赖图是一个有向无环图 $G = (V, E)$ ，其中：

- $V = V_{root} \cup V_{leaf} \cup V_{semi}$ is a set of nodes where:

$$V_{root} = \{v \mid \exists S_1 : v \in S_1.y \wedge \nexists S_2 : v \in S_2.X\}$$

$$V_{leaf} = \{v \mid \exists S_1 : v \in S_1.X \wedge \nexists S_2 : v \in S_2.y\}$$

$$V_{semi} = \{v \mid \exists S_1 \neq S_2 : v \in S_1.y \wedge v \in S_2.X\}$$

- $E = E_{RL} \cup E_{RS} \cup E_{SL} \cup E_{SS}$

- (1) V_{root} : 位于赋值语句左侧，且不被其他任何赋值语句所依赖的最终结果变量；
- (2) V_{leaf} : 只出现在等式右侧，不依赖于代码中的其他计算步骤，仅作为输入向图内提供数据；
- (3) V_{semi} : 在某些赋值语句中充当独立变量，在其他语句中充当依赖变量；
- (4) E_{RL} 是从 V_{root} 中的节点指向 V_{leaf} 中节点的有向边集合；
- (5) E_{RS} 表示从 V_{root} 到 V_{semi} 的边的集合；
- (6) E_{SL} 表示从 V_{semi} 到 V_{leaf} 的有向边；
- (7) E_{SS} 由连接 V_{semi} 内节点的边组成。

ADGFUZZ的原理-赋值依赖图的构建

赋值依赖图的构建：正向语法过滤+反向赋值分析。

- (1) 程序扫描整个函数，删除所有非赋值语句并对变量名进行处理，例如将 `func(var1)` 转换为变量名 `func_var1`；
- (2) 从**最后一行赋值语句开始向前倒推**，找到第一个还未被处理的左侧变量，将其标记为整个图的根节点；
- (3) 确定了根节点后，系统会继续往上看，找出所有参与计算该根节点的右侧变量，并在它们之间画上有向边；
- (4) 对于刚才找出的右侧变量，如果发现它在代码更上方的某个地方也作为左侧变量被赋过值，那么它就被定性为**半依赖节点**。如果它从头到尾只出现在等号右侧，它就是**叶子节点**；
- (5) 把新发现的半依赖节点当作当前节点，重复上述向上的追踪过程，直到没有任何新的依赖关系为止。在这个过程中，系统会自动删除可能导致死循环的重复关联；
- (6) 重复以上所有步骤，直到函数内所有的赋值语句都被解析并归入图中。

ADGFUZZ的原理-如何从ADG生成MIS

如何从ADG生成MIS：程序变量和系统输入参数通常遵循一致的命名规范（例如都包含相同的传感器缩写），并且它们在物理逻辑上是相互关联的。

1、将目标锁定在 ADG 的叶子节点 LVs上，这些节点代表了赋值链条中最源头的独立输入；

（1）提取叶子节点的变量名，并使用下划线作为分隔符，将变量名拆分成独立的词项（Variable terms）。同时也会对所有无人机支持的输入参数名进行同样的拆分操作；

（2）删除无语义的动词以及长度小于两个字符的无效短词。

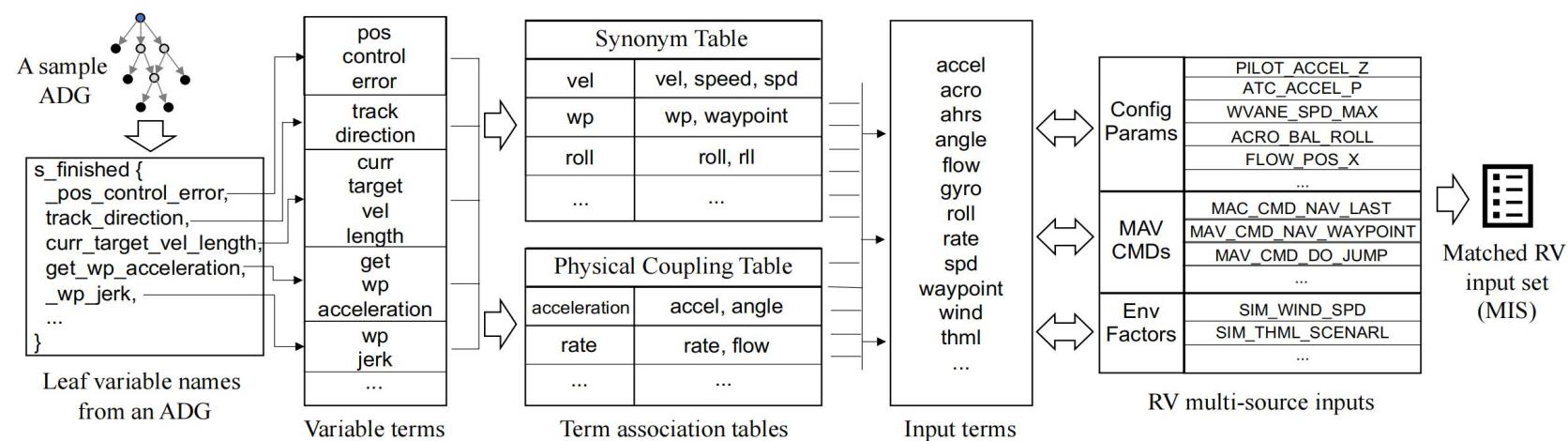


Fig. 4. An illustration of how MIS is inferred for a sample ADG based on term association.

ADGFUZZ的原理-如何从ADG生成MIS

- 2、为了解决代码变量名和输入参数名在拼写上不完全一致的问题，系统使用了：
- a.同义词表：处理同义词（例如location和position）以及代码中常见的缩写（例如将roll 缩写为 rll）；
 - b.物理耦合表：处理物理概念上的内在联系。例如，无人机的倾斜角度会直接影响其水平加速度，因此 angle 和 acceleration 会被视为一对关联词汇。这张表是通过大型语言模型分析源代码注释半自动生成的。

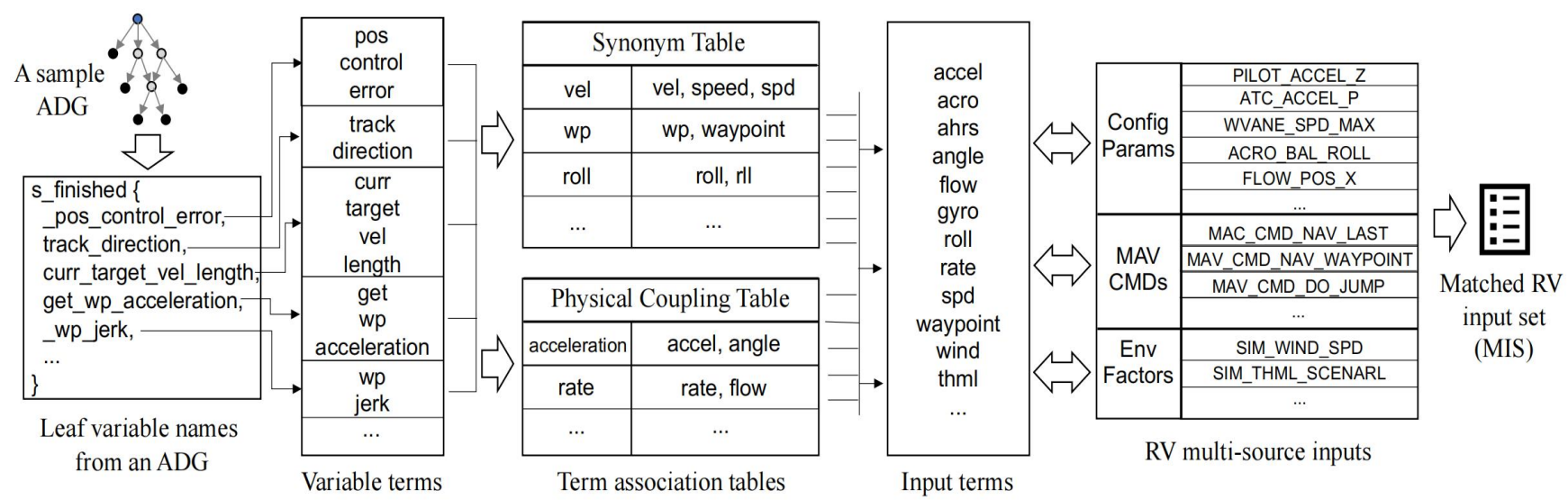


Fig. 4. An illustration of how MIS is inferred for a sample ADG based on term association.

ADGFUZZ的原理-如何从ADG生成MIS

- 3、经过上述扩展后，系统将转换后的代码变量词项与无人机输入词项进行严格的匹配对比。
- 4、只要某个无人机输入参数的词汇被成功匹配，该输入参数就会被拉入到最终的匹配输入集（MIS）中。同时，系统还会记录每个输入匹配成功的词项数量（这些数量数据将在后续阶段用于计算该 MIS 的信息熵，以决定模糊测试的优先级）。

通过上述操作，ADGFUZZ 就能把只存在于代码内部的赋值关系，精准地映射到真实世界中用户可以操控的配置参数或控制命令上。

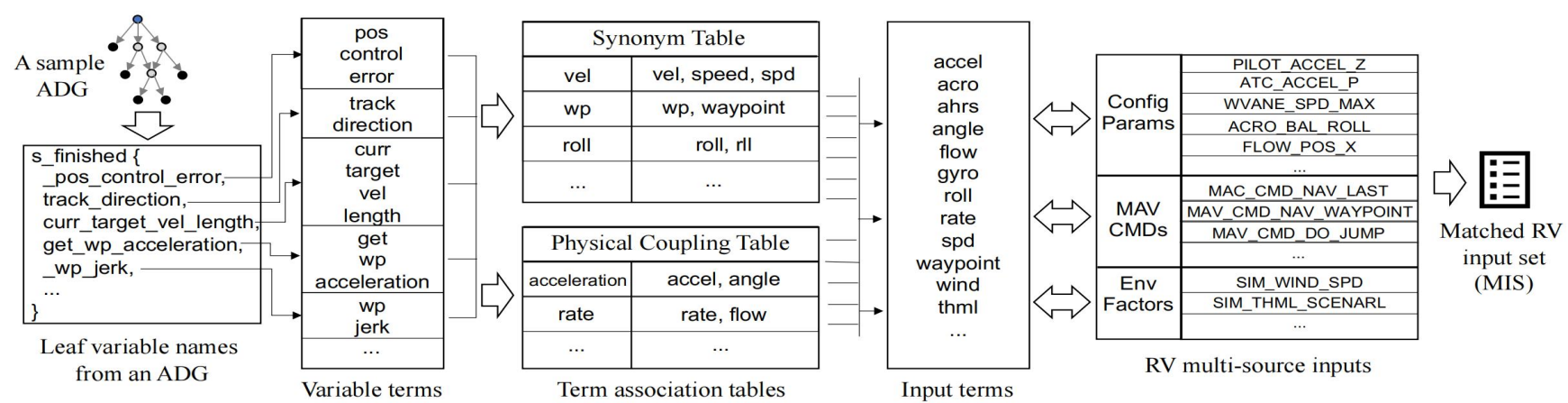


Fig. 4. An illustration of how MIS is inferred for a sample ADG based on term association.

ADGFUZZ的原理-基于熵的优先级排序

ADGFUZZ 为每个匹配输入集（MIS）定义了两个维度的信息熵：

1、结构复杂度熵 E_{num} ：衡量了底层控制逻辑的复杂程度。开发者通常不会编写大量无用的冗余赋值代码，因此，如果一个赋值依赖图（ADG）中包含越多的半依赖节点（即中间计算步骤），说明它背后的中间计算越丰富，潜在的语义信息量就越大，也就越容易隐藏逻辑漏洞。对于包含 N 个半依赖节点的 ADG， E_{num} 的计算公式为：

$$E_{\text{num}}(\text{MIS}) = \log_2(|N| + 1)$$

2、语义丰富度熵 E_{qual} ：评估了叶子节点（最源头的变量）与无人机实际输入参数之间的语义匹配质量。一个变量匹配到的有效词项越多，它携带的语义信息就越大。但是如果某个词项过于通用（例如匹配到了海量的不同无人机输入参数），它所能提供的特异性信息反而越少。假设变量 v 包含 k 个词项，使用其中的 i 个词项共同匹配到了 T_i 个唯一的无人机输入，那么该变量的有效信息熵 $M(v)$ 为：

$$M(v) = \sum_{i=1}^k \frac{i}{T_i}$$

ADGFUZZ的原理-基于熵的优先级排序

整个 MIS 的语义丰富度熵则是基于所有叶子节点 V_{leaf} 有效信息熵的累加。

将两者结合就得到了该 MIS 的总信息熵：

$$E(\text{MIS}) = \log_2(|N| + 1) + \log_2 \left(1 + \sum_{v \in V_{\text{leaf}}} \sum_{i=1}^k \frac{i}{T_i} \right) \quad p(\text{MIS}) = \frac{E(\text{MIS})}{\sum_{L \in \text{MISs}} E(L)}$$

因此系统并不会死板地执行：

- 1、系统将所有 MIS 的熵值转化为概率分布，总熵值越高的 MIS 被抽中生成测试用例的概率就越大；
- 2、熵值的绝对大小直接决定了当前轮次该 MIS 要执行测试的次数，因为高熵意味着输入变量多，需要更多次执行才能充分探索其内部的组合空间；
- 3、为了平衡深度与广度搜索，防止系统在某一个集合上死磕，系统采用了动态调整策略。如果一个 MIS 触发了 Bug，它的熵值会被直接减半，以限制对已发现 Bug 区域的过度挖掘；如果没有触发 Bug，它的熵值也会逐渐降低，降低其在后续轮次中被选中的概率，从而将算力让给其他未测试的低熵 MIS。

ADGFUZZ的原理-漏洞预言机

三个自动化的漏洞预言机：

- 1、坠地坠毁：空中飞行器由于动力失效或其他故障导致意外跌落到地面的情况；
- 2、航线偏离：监控正在执行航点导航任务的无人机是否持续偏离了设定的目标。系统采用了一个基于时间窗口的明确标准，如果无人机距离目的地的实际距离在一个短时间窗口内不仅没有缩短，反而持续增加，预言机就会将其分类为航线偏离异常；
- 3、软件崩溃：检测控制软件中与内存相关的底层错误，例如由于参数处理不当或缺少安全检查机制所引发的算术溢出或浮点溢出。预言机会监听系统的通信状态，如果连续 2 秒没有收到无人机发来的heartbeat messages，就会直接判定控制软件已经崩溃宕机。

通过这三个预言机，系统能够在模拟任务的各个阶段自动捕捉异常。

ADGFUZZ的原理-后处理

时间差：当我们向系统注入一个异常的测试输入时，无人机可能不会立刻崩溃，而是飞行了一段距离、或者累积了足够的误差后才真正表现出异常。因此，预言机在监测到漏洞时，系统记录下来的初始测试用例集C里面往往包含了从测试开始直到漏洞爆发所执行的所有输入，其中有大量是毫无关联的冗余操作。如何解决？

1、对于每一个存在嫌疑的测试集，系统会启动一个全新的模拟器，设定执行方向为正向，并从头开始依次注入测试用例。每次注入输入后，ADGFUZZ 会等待一段延迟时间（ τ ），让这个参数的物理效果在系统中充分蔓延传播。一旦某次注入后漏洞被触发，系统立刻更新当前的索引位置，并把这个触发用例抓取到精简集 M_i 中。随后系统会重启模拟器，仅仅把挑选出的 M_i 集合单独输入进去。如果漏洞重现，就说明 M_i 确实是最小触发集合。如果复现失败，说明漏洞的触发还依赖前面某些被忽略的前置条件。这时，系统会将执行方向反转为逆向，并重复上述步骤，直到找到完整的触发链条。

ADGFUZZ的原理-后处理

- 2、有些漏洞的潜伏期很长，系统会逐一从记录中移除输入项，然后一次次反复测试漏洞是否依然会发生。那些被删除后丝毫不影响漏洞触发的输入项，就会被安全删除。
- 3、因为程序的代码逻辑是相互交织的，不同的赋值依赖图（ADG）可能会包含重叠的变量节点，这就导致系统在测试不同的集合时，可能会用不同的路径触发了同一个漏洞。系统会把找出的最小集合按照漏洞类型进行分组。如果发现有两个完全相同的触发集合，系统会直接删除后发现的那个，只保留最早的记录。

ADGFUZZ: 代码分析

四个主要阶段:

1、数据准备与映射: 将源代码中的变量名、宏定义映射为无人机能理解的实际指令、参数或遥控器通道。

 Mapping.py: 将代码变量映射为飞控系统能识别的参数设置或 MAVLink 指令, 并为每条路径计算信息熵。

 constant.py: 规定在不同的无人机型 (Copter, Plane, Rover) 中, 各种操作分别对应遥控器的哪几个通道。

 parsefile.py: 读取一些外部的元数据文件, 比如环境变量列表、系统支持的完整 MAVLink 指令集等。

ADGFUZZ: 代码分析

四个主要阶段:

2、模糊测试: 启动仿真器 (SITL), 根据熵选择测试路径, 生成随机合法的参数值并发送给无人机。

adgfuzz.py: 解析命令行参数, 使用子进程拉起 ArduPilot 或 PX4 的 SITL (software in the loop) 仿真环境。

fuzz.py (ArduPilot) & fuzzpx4.py (PX4): 根据信息熵概率选择路径, 调度各个执行线程, 控制轮次, 并在发现 bug 时调整权重和重启仿真器。

runtimeDict.py: 为 MAVLink 指令或系统参数生成在合法范围内的随机测试数值。

rvmethod.py: 封装了大量常用的 MAVLink 协议指令 (获取位置、修改飞行模式、模拟遥控器输入、起飞、航点任务上传、参数修改等)。

ADGFUZZ: 代码分析

四个主要阶段:

3、状态监控: 持续监听无人机的状态数据, 通过oracles来判断无人机是否出现bug。

oracle.py: 定义了各种异常状态。

fuzz.py & fuzzpx4.py : 不断读取 bug_oracle 抛出的标志位来决定是否报警。

4、后处理: 当发现bug后, 利用最小化算法从输入中删除无关项, 找出触发bug的最小指令集。

postprocess.py: 采用二分法排错的思想, 通过不断丢弃部分测试指令并重新启动仿真器进行验证, 最终筛选出能触发该 bug 的最小指令集。